



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

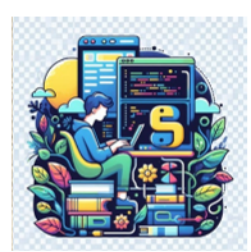
PROGRAMMING WITH PYTHON

Data Ingestion and Multiprocessing

Stream Data Ingestion

Lekha A

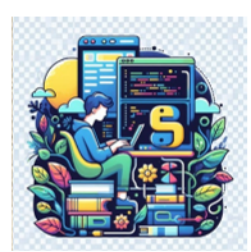
Computer Applications



PROGRAMMING WITH PYTHON

Stream Ingestion

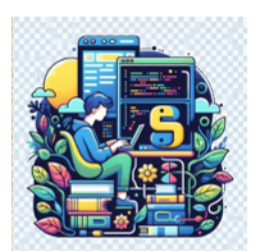
- In a multitude of applications, access to real-time and continuously updated data is of utmost importance.
- Fraud detection, churn prevention and recommendations are among the top candidates for streaming.



PROGRAMMING WITH PYTHON

Stream Ingestion

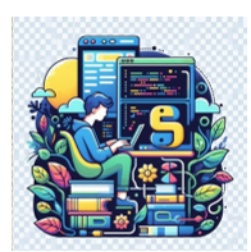
- These data pipelines handle the processing of data from various sources to multiple target destinations in real time, capturing events as they occur.
- They facilitate their transformation, enrichment and analysis.



PROGRAMMING WITH PYTHON

Data Pipeline

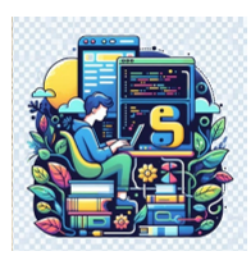
- A data pipeline refers to a series of steps involved in data processing.
 - The output from one stage serves as the input for the subsequent stage.
 - It establishes a coherent flow of data.
- A data pipeline is present whenever data is processed between two points, for instance, from a source to a destination.



PROGRAMMING WITH PYTHON

Data Pipeline

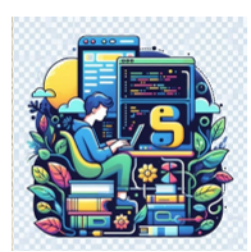
- The three primary components of a data pipeline include
 - the source
 - the processing step(s)
 - the destination.



PROGRAMMING WITH PYTHON

Data Pipeline

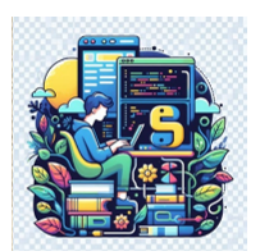
- For instance, data retrieved from an external API (source) can be transferred to a data warehouse (destination), exemplifying a typical scenario where the source and destination are separate.



PROGRAMMING WITH PYTHON

Data Pipeline

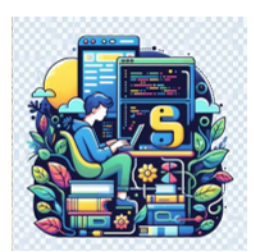
- Numerous Python frameworks exist to facilitate the development of data pipelines
 - Luigi
 - A Python package designed for creating complex pipelines of batch operations.
 - It effectively manages dependency resolution and aids in workflow management, simplifying the specification of jobs and their dependencies.



PROGRAMMING WITH PYTHON

Data Pipeline

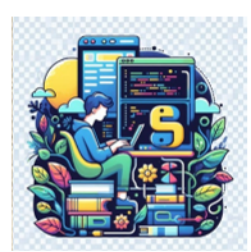
- Numerous Python frameworks exist to facilitate the development of data pipelines
 - Apache Beam
 - This framework offers a unified model that allows developers to construct data-parallel processing pipelines.
 - Beam accommodates both batch and streaming data, providing significant flexibility.
 - It is regarded as a versatile solution for addressing diverse data processing needs.



PROGRAMMING WITH PYTHON

Data Pipeline

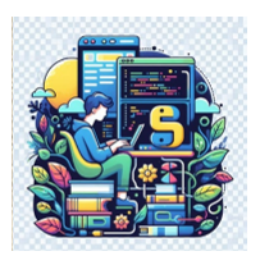
- Airflow
 - A structured platform for defining, scheduling, and monitoring workflows.
 - It enables the design of tasks and their dependencies, as well as the orchestration and oversight of processes.



PROGRAMMING WITH PYTHON

Data Pipeline

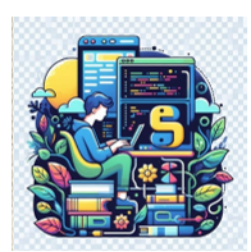
- Dask
 - A versatile Python module that simplifies the execution of parallel computing tasks.
 - It supports concurrent and larger-than-memory computations and integrates seamlessly with existing Python libraries such as Pandas and Scikit-Learn.



PROGRAMMING WITH PYTHON

Data Pipeline

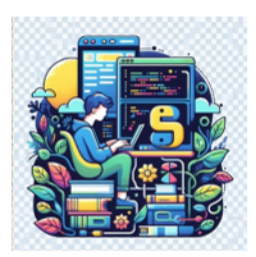
- Prefect
 - A workflow management system that enhances fault tolerance and streamlines the creation of data pipelines.
 - It provides a high-level, Python-centric interface for defining tasks and their dependencies.



PROGRAMMING WITH PYTHON

Terminologies

- Stream
 - A stream is treated as a continuous flow of events where an end is not assumed.
 - Example. Temperature readings sent every second from 50 sensors.



PROGRAMMING WITH PYTHON

Terminologies

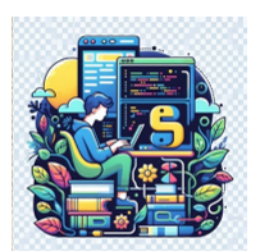
- Batch
 - A batch is treated as a fixed collection with a clear start and end.
 - Example. A CSV export of yesterday's readings.



PROGRAMMING WITH PYTHON

Terminologies

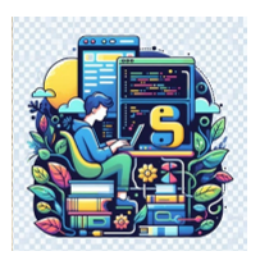
- Event
 - An event is treated as a single record that describes something that happened at a point in time.
- Example.
 - `sensor_id S01, ts 2025-12-28T10:01:00Z, temp_c 29.4, hum 62.1, seq`



PROGRAMMING WITH PYTHON

Terminologies

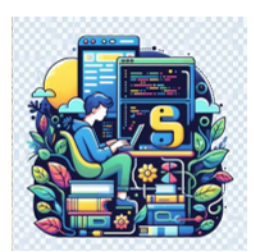
- Key
 - A key is treated as the field used to group or route events.
 - Example. `sensor_id` is used so all `S01` events can be aggregated together.
- Producer
 - A producer is the component from which events are emitted into a stream.
 - Example. `producer_file_stream.py` writes 1 event per 0.1 seconds into `events.jsonl`.



PROGRAMMING WITH PYTHON

Terminologies

- Consumer
 - A consumer is the component by which events are read and processed.
 - Example. `consumer_tail.py` reads new lines from `events.jsonl` and updates counts.



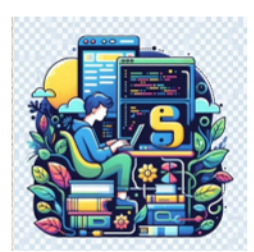
PROGRAMMING WITH PYTHON

Simulation of Stream Data Ingestion

- Create a producer_file_stream.py

Import Modules

```
import json
import random
import time
from datetime import datetime, timezone
```



PROGRAMMING WITH PYTHON

Simulation of Stream Data Ingestion

- ## Import Modules

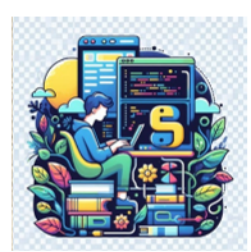
```
import json
import random
import time
from datetime import datetime, timezone
```

Create sensor Ids and Output file

JSONL text format is also referred to as newline-delimited JSON.

JSON Lines is an easy-to-use format for storing structured data that allows for record-by-record processing.

```
SENSORS = ["S01", "S02", "S03", "S04"]
OUT_FILE = "events.jsonl"
```



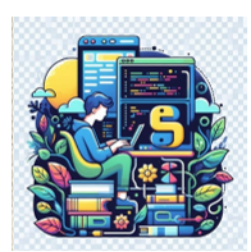
PROGRAMMING WITH PYTHON

Simulation of Stream Data Ingestion

Produce a timestamp in UTC.

```
def iso_utc_now():  
    #The current time is taken and UTC is forced ignoring the local system timezone  
    #isoformat - The datetime object is converted to text where only year, month, day, hour,  
    #minute, second are kept.  
    #replace - The UTC offset is replaced with Z where it represents Zulu time, which equals UTC.  
    return datetime.now(timezone.utc).isoformat(timespec="seconds").replace("+00:00", "Z")
```

- Zulu Time is used to avoid timezone confusion and avoid daylight saving errors.
- Logs from different countries align correctly.
- Events sort correctly as text.

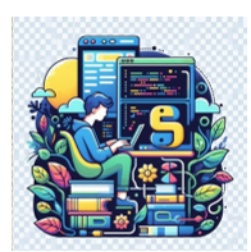


PROGRAMMING WITH PYTHON

Simulation of Stream Data Ingestion

- Create a single stream event

```
def make_event(sensor_id, seq):  
    temp_c = round(random.uniform(22.0, 36.0), 2)  
    hum = round(random.uniform(35.0, 80.0), 2)  
    #temperature and humidity is generated  
    return {  
        "sensor_id": sensor_id,  
        "ts": iso_utc_now(),  
        "temp_c": temp_c,  
        "hum": hum,  
        "seq": seq  
    }
```



PROGRAMMING WITH PYTHON

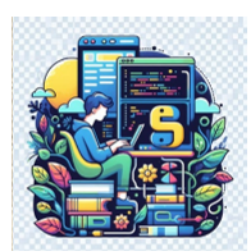
Simulation of Stream Data Ingestion

- A controlled stream of events is generated and written to a file

```
total_events = 1000
#The number of events to be produced is defined.
#This limits the stream length for testing.
delay_s = 0.1
#The delay between two consecutive events is defined in seconds.
#0.1 seconds means 10 events per second.
seq = 1
# A sequence counter is initialized. This uniquely identifies the order of events.
#It helps detect missing or duplicated events later.
with open(OUT_FILE, "a", encoding="utf-8") as f:
    for _ in range(total_events):
        sensor_id = random.choice(SENSORS)
        #One sensor ID is selected randomly.
        event = make_event(sensor_id, seq)
        #A complete event dictionary is created that includes sensor_id,
        #timestamp, temperature, humidity, and sequence number.
        f.write(json.dumps(event) + "\n")
        #The event dictionary is converted to a JSON string and a newline character is added.
        #Each line becomes one independent event.
        f.flush()
        #The write buffer is forced to disk immediately. The consumer can read the event without delay.
        #This mimics real time streaming behavior.
        seq += 1
        #The sequence number is incremented. The next event will have a higher order value.
        time.sleep(delay_s)
        #The program pauses before producing the next event.

print(total_events, "events written to", OUT_FILE)
```

1000 events written to events.jsonl



PROGRAMMING WITH PYTHON

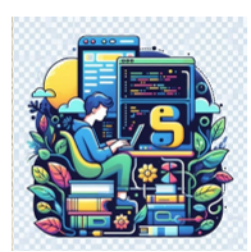
Simulation of Stream Data Ingestion

- Create a consumer_tail.py
- **Import the modules**

```
import json
import time
from collections import defaultdict
```

Input File

```
IN_FILE = "events.jsonl"
```

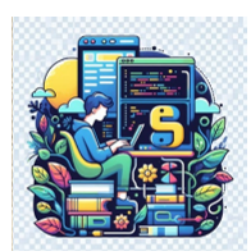


PROGRAMMING WITH PYTHON

Simulation of Stream Data Ingestion

Simulate a live stream.

```
def follow(file_obj):  
    #A generator function is defined.  
    file_obj.seek(0, 2)  
    #The file pointer is moved to the end of the file.  
    #Only new data is processed.  
    while True:  
        line = file_obj.readline()  
        #One line is read from the file.  
        if not line:  
            #This condition checks for no new data.  
            time.sleep(0.1)  
            #The loop pauses briefly.  
            continue  
            #Control jumps back to the loop start.  
        yield line
```

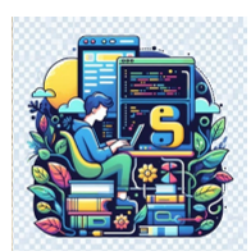



PROGRAMMING WITH PYTHON

Simulation of Stream Data Ingestion

A live stream is consumed.

```
counts = defaultdict(int)
#A dictionary is created for counting.
#Missing keys are given value zero automatically.
total = 0
#A counter is created for all processed events.
bad_json = 0
#A counter is created for corrupted records.
MAX_EVENTS = 100
#to limit the consumer
```



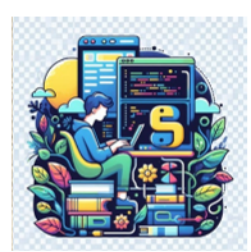
PROGRAMMING WITH PYTHON

Simulation of Stream

```
with open(IN_FILE, "r", encoding="utf-8") as f:
    #The stream file is opened in read mode and UTF-8 encoding is enforced.
    for line in follow(f):
        #New lines are consumed as they appear.
        #The loop never ends by design.
        #Streaming behavior is achieved.
        line = line.strip()
        #Whitespace and newline characters are removed.
        if not line:
            #Empty lines are ignored and noise is filtered
            continue
        try:
            event = json.loads(line)
            #The text line is converted to a Python dictionary.
        except json.JSONDecodeError:
            #Malformed JSON is detected
            bad_json += 1
            #bad_json is incremented.
            continue

        sensor_id = event.get("sensor_id", "UNKNOWN")
        #The event key is extracted. UNKNOWN is used if the field is missing.
        counts[sensor_id] += 1
        #The event count for this sensor is increased.
        total += 1
        #The global event count is increased.

        if total % 20 == 0:
            #Reporting is triggered every 20 events.
            print("Total:", total, "Bad JSON:", bad_json, "Counts:", dict(counts))
        if total >= MAX_EVENTS:
            print("Limit reached. Stopping consumer")
            break
```



PROGRAMMING WITH PYTHON

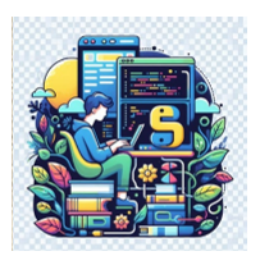
Simulation of Stream Data Ingestion

- Run both the files in terminals simultaneously

```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\StreamData> python producer_file_stream.py
```

```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\StreamData> python consumer_tail.py
Total: 20 Bad JSON: 0 Counts: {'S01': 6, 'S04': 5, 'S03': 6, 'S02': 3}
Total: 40 Bad JSON: 0 Counts: {'S01': 9, 'S04': 9, 'S03': 14, 'S02': 8}
Total: 60 Bad JSON: 0 Counts: {'S01': 15, 'S04': 12, 'S03': 20, 'S02': 13}
Total: 80 Bad JSON: 0 Counts: {'S01': 25, 'S04': 16, 'S03': 22, 'S02': 17}
Total: 100 Bad JSON: 0 Counts: {'S01': 30, 'S04': 20, 'S03': 29, 'S02': 21}
Limit reached. Stopping consumer
```

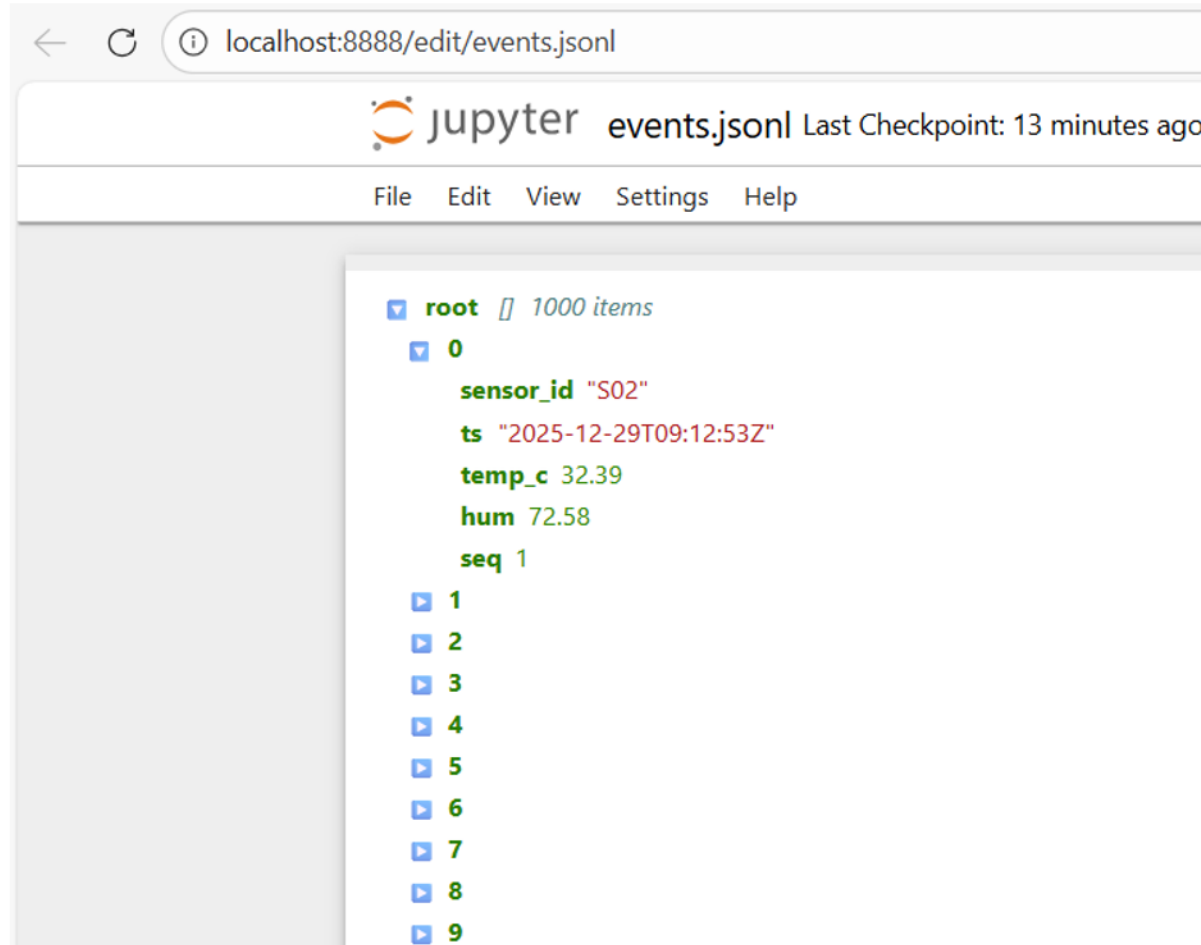
```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\StreamData> python producer_file_stream.py
1000 events written to events.jsonl
```



PROGRAMMING WITH PYTHON

Simulation of Stream Data Ingestion

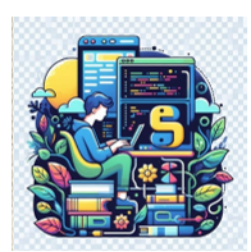
- Output



The screenshot shows a web browser at `localhost:8888/edit/events.jsonl` displaying a Jupyter Notebook. The notebook has a menu bar with `File`, `Edit`, `View`, `Settings`, and `Help`. The main content area shows a JSON array with 1000 items. The first item (index 0) is expanded, showing the following fields:

```
sensor_id "S02"  
ts "2025-12-29T09:12:53Z"  
temp_c 32.39  
hum 72.58  
seq 1
```

Below the first item, indices 1 through 9 are listed, each with a small blue square icon to its left, indicating they are collapsed.



PROGRAMMING WITH PYTHON

Recap

- Stream Ingestion
- Data Pipeline
- Terminologies
- Simulation of Stream Ingestion



THANK YOU

Lekha A
Computer Applications